

Script de Apresentação

Script de Apresentação — Docker & Docker Compose (12 min + 2 min Q&A)

Abertura (1 min)

Tempo	Fala	Tela
0:00–0:15	“Olá, nosso grupo vai apresentar Docker e Docker Compose — ferramentas essenciais para containers e entrega de aplicações.”	Slide 1 — Capa
0:15–0:30	“Vamos começar com um problema comum: quantos aqui já ouviram ‘funciona na minha máquina’? Ou já perderam horas configurando ambiente em um projeto novo?”	Slide 2 — O Problema
0:30–1:00	“Docker resolve exatamente isso. Ele empacota sua aplicação com todas as dependências em um container leve e isolado. O resultado: roda igual no seu notebook, no servidor da faculdade e na nuvem.”	Slide 2 — O Problema

O que é Docker? (1 min)

Tempo	Fala	Tela
1:00–1:20	“Docker é uma plataforma de containers. Um container é como uma máquina virtual, mas muito mais leve — compartilha o kernel do sistema operacional do host.”	Slide 3 — Docker: Conceitos
1:20–1:40	“Temos dois conceitos principais: imagem e container . A imagem é um template imutável — pense como uma ISO. O container é uma instância executável dessa imagem.”	Slide 3 — Docker: Conceitos
1:40–2:00	“A imagem é construída a partir de um Dockerfile , que é uma receita declarando o ambiente. E as imagens ficam armazenadas em registries como o Docker Hub.”	Slide 3 — Docker: Conceitos

O que é Docker Compose? (1 min)

Tempo	Fala	Tela
2:00–2:20	“Em projetos reais, você nunca roda apenas a aplicação. Precisa de banco de dados, cache, fila de mensagens, proxy reverso. Gerenciar tudo manualmente é inviável.”	Slide 4 — Docker Compose
2:20–2:40	“Docker Compose resolve isso com um arquivo YAML declarando todos os serviços. Um comando — <code>docker compose up -d</code> — sobe a stack inteira.”	Slide 4 — Docker Compose
2:40–3:00	“Outro comando — <code>docker compose down</code> — derruba tudo de forma limpa. E com <code>-v</code> remove também os volumes de dados.”	Slide 4 — Docker Compose

Docker em Sistemas para Internet (2 min)

Tempo	Fala	Tela
3:00–3:15	“Agora, por que Docker é essencial para Sistemas para Internet? Vou destacar três pontos principais.” ***“Primeiro: paridade entre desenvolvimento e produção.** O mesmo container que você sobe no seu computador com docker compose up vai	Slide 5 — Sistemas para Internet
3:15–3:45	exatamente igual para o servidor de produção. Nada de configurações diferentes, versões diferentes de bibliotecas, ‘mas na minha máquina funciona’. É o mesmo ambiente, ponto final.” ***“Segundo: onboarding de novos desenvolvedores.** Em uma equipe, quando chega um dev novo, o tutorial de setup costumava ser um documento de 20 páginas — instale PostgreSQL, Node, Redis, configure variáveis, etc. Com Docker, o tutorial vira: ‘instale Docker e rode docker compose up’. Minutos, não dias.”	Slide 5 — Sistemas para Internet
3:45–4:15	***“Terceiro: microsserviços e isolamento.** Cada serviço da sua aplicação — autenticação, pagamento, catálogo, banco — roda em seu próprio container. Cada um escala de forma independente. Cada um pode ter sua própria versão de linguagem sem conflitar com os outros.”	Slide 5 — Sistemas para Internet
4:15–4:45	“E um bônus: CI/CD. Pipelines de integração contínua sobem	Slide 5 — Sistemas para Internet
4:45–5:00	containers idênticos aos de produção para testar, e depois promovem a mesma imagem para produção.”	Slide 5 — Sistemas para Internet

Instalação (1 min)

Tempo	Fala	Tela
5:00–5:15	“Vamos ver rapidamente como instalar.”	Slide 6 — Instalação
5:15–5:30	“No Linux, Ubuntu/Debian, o comando é <code>sudo apt install docker.io</code> . Nas outras distribuições ou se quiser a versão mais recente, use o script oficial da Docker: <code>curl -fsSL https://get.docker.com sh</code> ”	Slide 6 — Instalação
5:30–5:50	“Após instalar, um passo obrigatório : adicionar seu usuário ao grupo <code>docker</code> com <code>sudo usermod -aG docker \$USER</code> e depois <code>newgrp docker</code> ou relogar. Isso evita ter que usar <code>sudo</code> em todo comando.”	Slide 6 — Instalação
5:50–6:00	“Para verificar se deu certo: <code>docker --version</code> e <code>docker run hello-world</code> . Essa imagem vai baixar e rodar um container de teste.”	Slide 6 — Instalação

Comandos Essenciais (1 min)

Tempo	Fala	Tela
6:00–6:20	“Comandos principais do Docker: para imagens usamos <code>docker pull</code> , <code>docker build</code> , <code>docker images</code> , <code>docker rmi</code> . Para containers: <code>docker run</code> , <code>docker ps</code> , <code>docker stop</code> , <code>docker rm</code> , <code>docker exec</code> e <code>docker logs</code> .”	Slide 7 — Comandos Essenciais
6:20–6:40	“Flags importantes do <code>docker run</code> : <code>-d</code> para rodar em background, <code>-p</code> para mapear porta, <code>-v</code> para volume, <code>-e</code> para variáveis de ambiente, <code>--name</code> para nomear o container.”	Slide 7 — Comandos Essenciais
6:40–7:00	“Para Compose: <code>docker compose up -d</code> para subir, <code>docker compose down -v</code> para derrubar com volumes, <code>docker compose logs -f</code> para acompanhar logs, <code>docker compose exec</code> para executar comandos em serviços.”	Slide 7 — Comandos Essenciais

Caso Real: Arquitetura + Demo (3 min)

Tempo	Fala	Tela
7:00–7:15	“Agora vamos para a prática. Criamos um repositório no GitHub com uma aplicação Node.js + PostgreSQL usando Docker Compose.”	Slide 8 — Caso Real: Arquitetura
7:15–7:30	“A arquitetura é simples: dois serviços em rede isolada. O app se conecta ao banco pelo nome do serviço db. O Dockerfile usa node:20-alpine, leve. O compose.yaml declara serviços com healthcheck e volume nomeado.”	Slide 8 — Caso Real: Arquitetura
7:30–7:45	“Agora a demo. Primeiro, clonamos o repositório e entramos na pasta.”	Slide 9 — Caso Real: Demo / Terminal: git clone
7:45–8:15	“O comando mágico: docker compose up -d --build. Ele baixa a imagem do PostgreSQL, constrói a imagem da aplicação, cria a rede e os volumes, e sobe tudo.”	Terminal: docker compose up -d --build
8:15–8:30	“Com docker compose ps vemos os dois serviços rodando. App na porta 3000, db na 5432.”	Terminal: docker compose ps
8:30–8:45	“Testamos com curl http://localhost:3000. A aplicação consulta o banco e retorna um JSON com a hora atual. A stack inteira está funcionando.”	Terminal: curl, mostrar JSON
8:45–9:00	“Vamos ver os logs com docker compose logs --tail=10.”	Terminal: docker compose logs
9:00–9:30	“Para encerrar a demo: docker compose down -v. Remove containers, redes e volumes. Tudo limpo.”	Terminal: docker compose down -v

Erros Comuns e Práticas (1 min)

Tempo	Fala	Tela
10:00–10:15	“Dois erros que todo mundo enfrenta no início. Primeiro: ‘Cannot connect to the Docker daemon’ — é o usuário sem permissão no grupo docker. Basta rodar <code>sudo usermod -aG docker \$USER</code> .”	Slide 10 — Erros Comuns
10:15–10:30	“Segundo: ‘port is already allocated’ — já tem algo na porta. Use <code>docker ps</code> e <code>lsof</code> pra diagnosticar, e mude a porta com <code>-p</code> .”	Slide 10 — Erros Comuns
10:30–10:45	“Boas práticas: use imagens Alpine, tenha <code>.dockerignore</code> , copie <code>package.json</code> antes do código, nomeie volumes e use <code>healthcheck</code> no Compose.”	Slide 11 — Boas Práticas
10:45–11:00	“O guia rápido que preparamos tem todos esses detalhes, incluindo 4 erros comuns com soluções e referências oficiais.”	Slide 11 — Boas Práticas

Encerramento (1 min)

Tempo	Fala	Tela
11:00–11:15	“Recapitulando: vimos o que é Docker e Docker Compose, por que são essenciais para Sistemas para Internet, os comandos principais e uma demo funcionando na prática.”	Slide 12 — Perguntas
11:15–11:30	“O repositório com o código da demo está em github.com/VDCSS/demo-Docker . O guia completo está disponível em PDF.”	Slide 12 — Perguntas
11:30–12:00	“Referências oficiais: docs.docker.com , Dockerfile reference, Compose file reference, Docker Hub. Todas no guia. Obrigado! Agora estamos abertos para perguntas.”	Slide 12 — Perguntas

Q&A (2 min)

Tempo	Preparação
12:00–12:30	Possível pergunta: “Qual a diferença entre docker-compose (hífen) e docker compose (espaço)?” — Resposta: docker-compose é o script Python legado. docker compose é o plugin moderno, integrado ao Docker CLI desde 2023. Use sempre o segundo.
12:30–13:00	Possível pergunta: “Precisa instalar Node.js e PostgreSQL no computador?” — Resposta: Não. Tudo roda dentro dos containers. As imagens oficiais do Node e PostgreSQL já vêm com tudo configurado. Você só precisa do Docker.
13:00–13:30	Possível pergunta: “Como faço para acessar o banco de fora do container?” — Resposta: Por padrão, a porta 5432 não é exposta. Se precisar, adicione ports: ["5432:5432"] no service db do compose.yaml.
13:30–14:00	Possível pergunta: “Docker é só para Linux?” — Resposta: Não. No Windows e Mac, o Docker Desktop cria uma VM Linux leve por baixo. Os comandos e a experiência são idênticos.

Anexo: Comandos da Demo

```
# 1. Clonar
git clone https://github.com/VDCSS/demo-Docker.git
cd demo-Docker

# 2. Build e subir
docker compose up -d --build

# 3. Verificar status
docker compose ps

# 4. Testar
curl http://localhost:3000
# {"status":"ok","time":"2026-06-13T..."}

# 5. Ver logs
docker compose logs --tail=10

# 6. Executar comando no banco
docker compose exec db psql -U postgres -d meubanco -c "SELECT NOW();"

# 7. Derrubar
docker compose down -v
```